BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 6

Roll no: 66 PRN No: 12411609 Name of Student: Laukika Shinde

Subject Name: Deep Learning

Problem Statement: Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Algorithm:

1) Import the required libraries.
2) Download and extract the Soybean Disease dataset.
3) Load the images and assign disease labels based on their folders.
4) Resize and normalize the images.
5) Split the dataset into training and validation sets.
6) Build a CNN using convolution, pooling, flatten, and dense layers.
7) Compile the CNN using Adam optimizer and cross-entropy loss.
8) Train the CNN on the soybean images.
9) Evaluate the model using validation accuracy and loss.
10) Plot accuracy and loss graphs and predict disease classes.

Code:

```
!pip -q install kaggle


from google.colab import files


print("Upload your kaggle.json file:")

uploaded = files.upload()
```

```
!mkdir -p ~/.kaggle

!cp kaggle.json ~/.kaggle/

!chmod 600 ~/.kaggle/kaggle.json


!kaggle datasets download -d sivm205/soybean-diseased-leaf-dataset

import zipfile

import os


zip_file = "/content/soybean-diseased-leaf-dataset.zip"


with zipfile.ZipFile(zip_file, 'r') as zip_ref:

    zip_ref.extractall("/content/soybean")


!find /content/soybean -maxdepth 2 -type d


# The following searches for the directory containing

# the actual disease-class folders.


for root, dirs, files in os.walk("/content/soybean"):

    if len(dirs) > 1:

        print("Possible dataset directory:", root)

        print("Classes:", dirs[:10])


DATASET_PATH = "/content/soybean"
```

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras import layers, models

IMG_SIZE = (128, 128)
BATCH_SIZE = 32
SEED = 123

train_ds = tf.keras.utils.image_dataset_from_directory(
    DATASET_PATH,
    validation_split=0.2,
    subset="training",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    DATASET_PATH,
    validation_split=0.2,
    subset="validation",
```

```
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

class_names = train_ds.class_names

print("\nDisease Classes:")
print(class_names)

print("\nNumber of Classes:", len(class_names))

plt.figure(figsize=(10, 10))

for images, labels in train_ds.take(1):

    for i in range(min(9, len(images))):

        plt.subplot(3, 3, i + 1)

        plt.imshow(images[i].numpy().astype("uint8"))

        plt.title(class_names[labels[i]])
```

```
        plt.axis("off")

plt.show()

normalization_layer = layers.Rescaling(1./255)

train_ds = train_ds.map(
    lambda x, y: (normalization_layer(x), y)
)

val_ds = val_ds.map(
    lambda x, y: (normalization_layer(x), y)
)
]

num_classes = len(class_names)

model = models.Sequential([

    # Convolution Layer 1
    layers.Conv2D(
        32,
        (3, 3),
        activation='relu',
```

```
    input_shape=(128, 128, 3)
),

layers.MaxPooling2D((2, 2)),


# Convolution Layer 2
layers.Conv2D(
    64,
    (3, 3),
    activation='relu'
),

layers.MaxPooling2D((2, 2)),


# Convolution Layer 3
layers.Conv2D(
    128,
    (3, 3),
    activation='relu'
),

layers.MaxPooling2D((2, 2)),
```

```
    # Flatten
    layers.Flatten(),


    # Fully Connected Layer
    layers.Dense(
        128,
        activation='relu'
    ),


    # Output Layer
    layers.Dense(
        num_classes,
        activation='softmax'
    )
])

model.summary()

model.compile(
    optimizer='adam',
```

```
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10
)
loss, accuracy = model.evaluate(val_ds)

print("\nValidation Loss:", loss)

print(
    "Validation Accuracy:",
    round(accuracy * 100, 2),
    "%"
)
plt.figure(figsize=(8, 5))

plt.plot(
    history.history['accuracy'],
    label='Training Accuracy'
)
```

```
plt.plot(
    history.history['val_accuracy'],
    label='Validation Accuracy'
)

plt.xlabel("Epoch")

plt.ylabel("Accuracy")

plt.title("Training and Validation Accuracy")

plt.legend()

plt.show()
plt.figure(figsize=(8, 5))

plt.plot(
    history.history['loss'],
    label='Training Loss'
)

plt.plot(
    history.history['val_loss'],
```

```
    label='Validation Loss'
)

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.title("Training and Validation Loss")

plt.legend()

plt.show()
print("\nUpload a soybean leaf image for prediction:")

uploaded_image = files.upload()

image_path = next(iter(uploaded_image))



# Load Image
img = tf.keras.utils.load_img(
    image_path,
    target_size=IMG_SIZE
)
```

```
# Convert Image to Array
img_array = tf.keras.utils.img_to_array(img)



# Normalize
img_array = img_array / 255.0



# Add Batch Dimension
img_array = np.expand_dims(
    img_array,
    axis=0
)
prediction = model.predict(img_array)

predicted_class = class_names[
    np.argmax(prediction)
]

confidence = np.max(prediction) * 100
```

```
print("\nPredicted Disease:", predicted_class)


print(
    "Confidence:",
    round(confidence, 2),
    "%"
)


]


plt.figure(figsize=(5, 5))


plt.imshow(img)


plt.title(
    f"Prediction: {predicted_class}\n"
    f"Confidence: {confidence:.2f}%"
)


plt.axis("off")


plt.show()


#Prediction module
```

```
from google.colab import files as colab_files

print("Upload a soybean leaf image:")

uploaded_image = colab_files.upload()

image_path = next(iter(uploaded_image))

# Load image
img = tf.keras.utils.load_img(
    image_path,
    target_size=IMG_SIZE
)

# Convert to array
img_array = tf.keras.utils.img_to_array(img)

# Normalize
img_array = img_array / 255.0

# Add batch dimension
img_array = np.expand_dims(img_array, axis=0)

# Prediction
```

```
prediction = model.predict(img_array)


predicted_class = class_names[np.argmax(prediction)]
confidence = np.max(prediction) * 100


print("Predicted Disease:", predicted_class)
print("Confidence:", round(confidence, 2), "%")


# Display image
plt.figure(figsize=(5, 5))
plt.imshow(img)
plt.title(
    f"Prediction: {predicted_class}\n"
    f"Confidence: {confidence:.2f}%"
)
plt.axis("off")
plt.show()
```

Output:

Sudden Death Syndrone

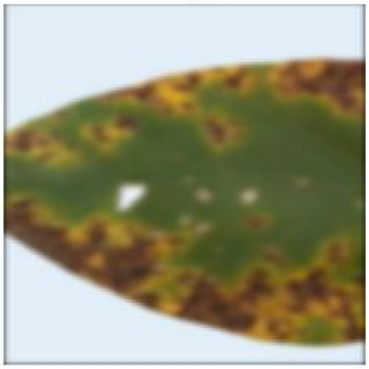
Yellow Mosaic

Yellow Mosaic

bacterial_blight

Southern blight

brown_spot



Yellow Mosaic

brown_spot

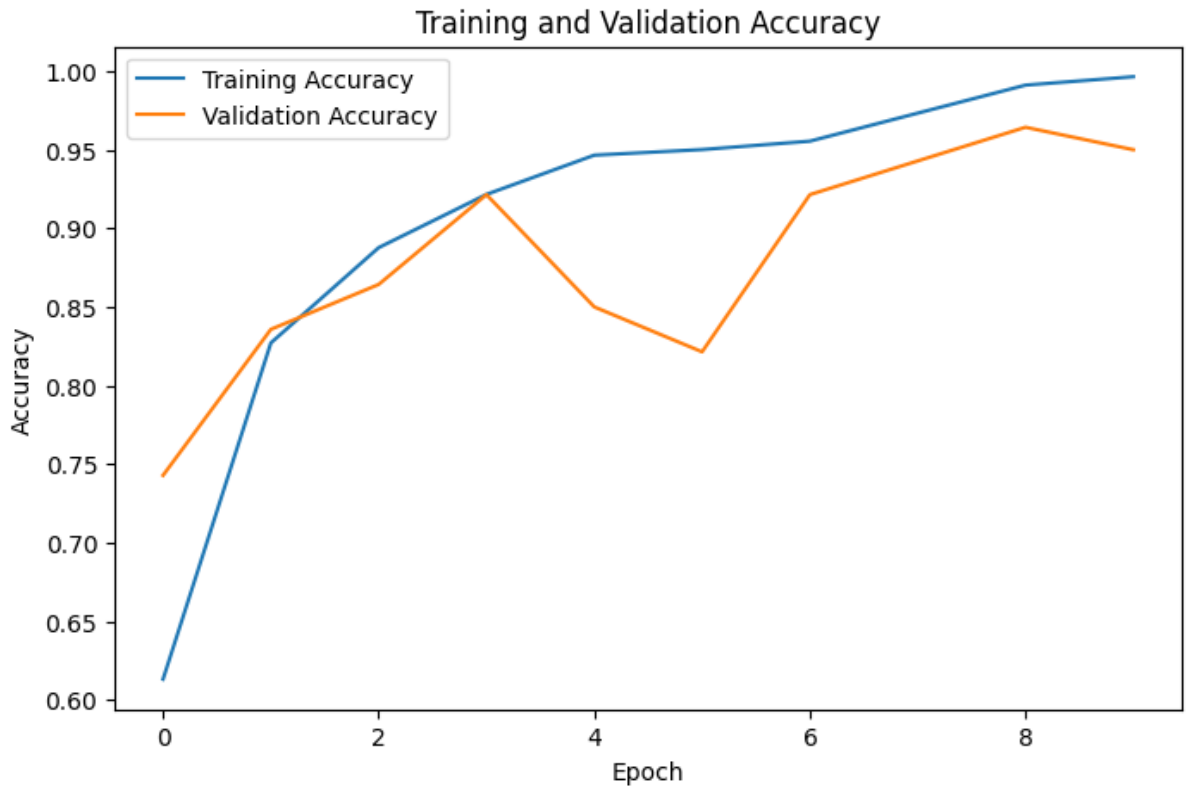
ferrugen

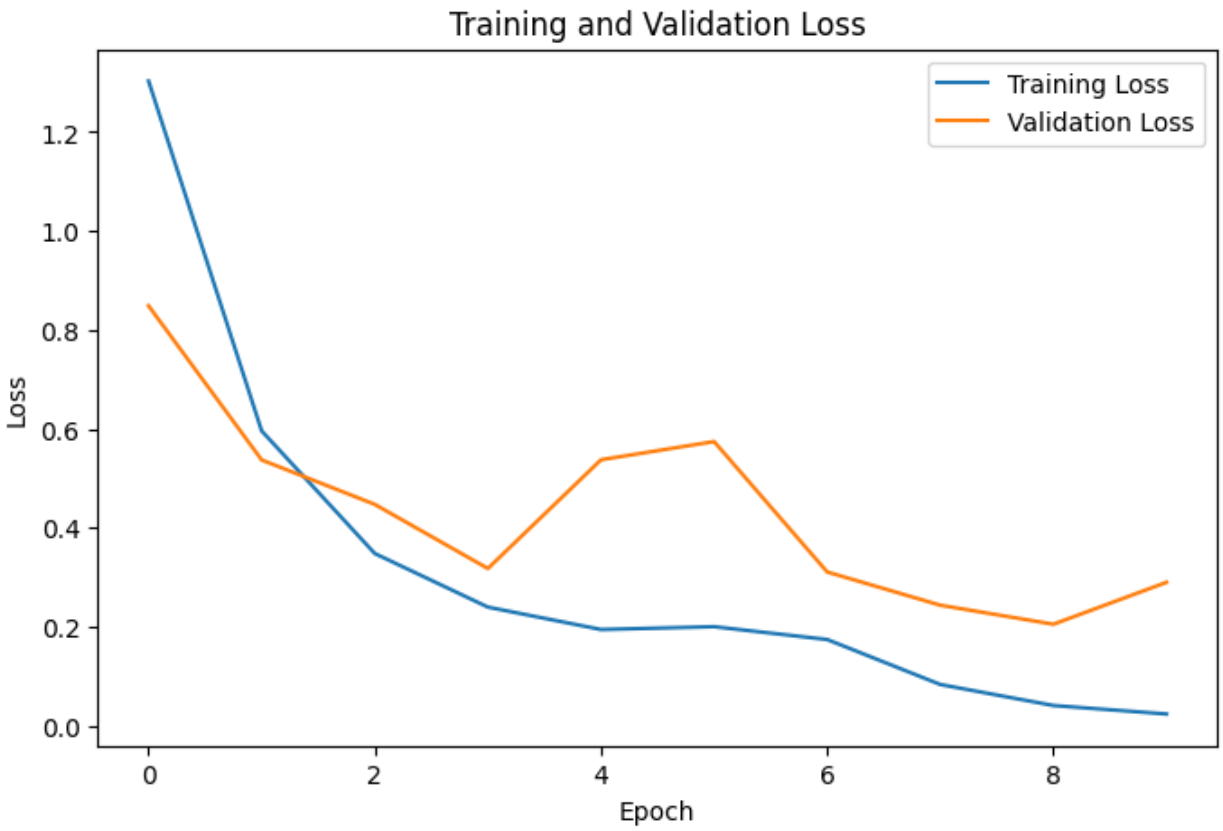
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 128)	3,211,392
dense_1 (Dense)	(None, 10)	1,290

Total params: 3,305,930 (12.61 MB)
Trainable params: 3,305,930 (12.61 MB)
Non-trainable params: 0 (0.00 B)

Validation Loss: 0.2902657687664032

Validation Accuracy: 95.0 %


Training and Validation Accuracy
Training Accuracy
Validation Accuracy
Accuracy
Epoch
1.00
0.95
0.90
0.85
0.80
0.75
0.70
0.65
0.60
0
2
4
6
8


Training and Validation Loss
Training Loss
Validation Loss
Loss
Epoch
1.2
1.0
0.8
0.6
0.4
0.2
0.0
0
2
4
6
8

Predicted Disease: brown_spot

Confidence: 58.26 %